Academic Management

(Layanan Jadwal Kuliah dan Ruang: Jadwal Pribadi, Perubahan Jadwal, Informasi Ruang, dan Bentrok Jadwal)



Dosen Pengampu:

Mardiyyah Hasnawi, S. Kom., M.T.

Disusun Oleh

Kelompok 6:

1. Fajar (13020240083)
2. Hasyim Muzadi (13020240099)
3. Ashadul Muslihun (13020240351)
4. Dwi Andhika Putra (13020240119)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MUSLIM INDONESIA

MAKASSAR

2026

DAFTAR ISI

DAFTAR ISI 2
BAB I PENDAHULUAN 4
U1 Konteks dan ASR 4
1.1 Ringkasan Layanan 4
1.2 Masalah Utama Layanan dan Bukti Empiris 4
1.3 Stakeholder Register dan Matriks Power/Interest 6
1.4 Architectural Significance Requirements (ASR) 7
1.5 Karakteristik Pendorong (Architectural Drivers) 7
BAB II SKENARIO 9
U2. Skenario Atribut Kualitas 9
2.1 Pendahuluan 9
2.2 Skenario Atribut Kualitas 9
2.3 Kesimpulan 10
BAB III TAKTIK DAN POLA 11
U3. TAKTIK ARSITEKTUR (Architectural Tactics) 11
3.1 Taktik Skenario 1: Performance Efficiency (Performa & Akurasi Deteksi Bentrok) 11
3.2 Taktik Skenario 2: Usability (Kemudahan Akses Dasbor & Informasi Jadwal) 11
3.3 Taktik Skenario 3: Reliability (Keandalan Pencatatan & Sinkronisasi Lintas Modul) 12
U4. POLA ARSITEKTUR (Architectural Patterns) 13
3.1 Pola Utama yang Digunakan: *Layered Architecture* (N-Tier Architecture) dengan *Microservices-ready Modules* 13
3.2 Kesesuaian Pola Arsitektur dengan Karakteristik Pendorong (ASR) 13
BAB IV DESAIN 15
U5. DESAIN ARSITEKTUR 15
4.1 Struktur Pandangan Arsitektur (*Architectural Views*) 15
4.2 Diagram Konteks (C4 System Context - Level 1) 16
BAB V EVALUASI DAN ADR 17

U6. EVALUASI ARSITEKTUR 17
6.1 Pohon Utilitas Mini (*Mini Utility Tree*) 17
6.2 Matriks Evaluasi Trade-off Berbobot (*Weighted Trade-off Evaluation Matrix*) 17
6.3 Analisis ATAM (Architecture Trade-off Analysis Method) 18
6.4 Keterbatasan Masalah Utama 19
U7. ARCHITECTURAL DECISION RECORDS (ADR) 20
BAB VI PENUTUP 23
A. Daftar Pustaka 23

BAB I
PENDAHULUAN

U1 Konteks dan ASR

1.1 Ringkasan Layanan

Layanan *Academic Management* (khususnya sub-modul Jadwal Kuliah dan Ruang) merupakan salah satu layanan inti pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI). Layanan ini bertanggung jawab dalam mengelola seluruh siklus alokasi perkuliahan, yang meliputi:

- Pembuatan dan penerbitan jadwal perkuliahan mingguan.
- Pengelolaan dan alokasi ruang kelas fisik beserta fasilitas penunjangnya.
- Pengaturan dan validasi jam mengajar dosen.
- Pengajuan dan persetujuan perubahan jadwal (*rescheduling*) secara dinamis.
- Penayangan dasbor jadwal pribadi (*personal schedule dashboard*) yang tersesuaikan untuk mahasiswa dan dosen.

1.2 Masalah Utama Layanan dan Bukti Empiris

A. Masalah Utama

Pengelolaan jadwal perkuliahan pada SIAKAD saat ini berpotensi tinggi memicu bentrok jadwal mata kuliah, ruang kelas, dan waktu mengajar dosen, serta ketidaksesuaian kapasitas ruang dengan jumlah mahasiswa. Hal ini terjadi karena proses verifikasi ketersediaan ruang dan jadwal dosen belum terintegrasi secara otomatis dan sinkron saat pengajuan perubahan maupun penyusunan awal (sesuai ADR-001). Faktor pemicu yang teridentifikasi dari para pemangku kepentingan meliputi mahasiswa yang mengambil mata kuliah lintas semester (ke atas atau ke bawah), keterbatasan jumlah ruang kelas, jadwal laboratorium yang tidak sinkron dengan jadwal teori, serta data ruang di sistem digital yang tidak sesuai dengan kondisi fisik di lapangan. Akibatnya, mahasiswa dan dosen sering kali baru menyadari terjadinya *overlapping* setelah jadwal dipublikasikan, sehingga jadwal harus diubah berulang kali dan proses koreksi dilakukan secara manual, memakan waktu lama, dan mengganggu kelancaran perkuliahan awal semester.

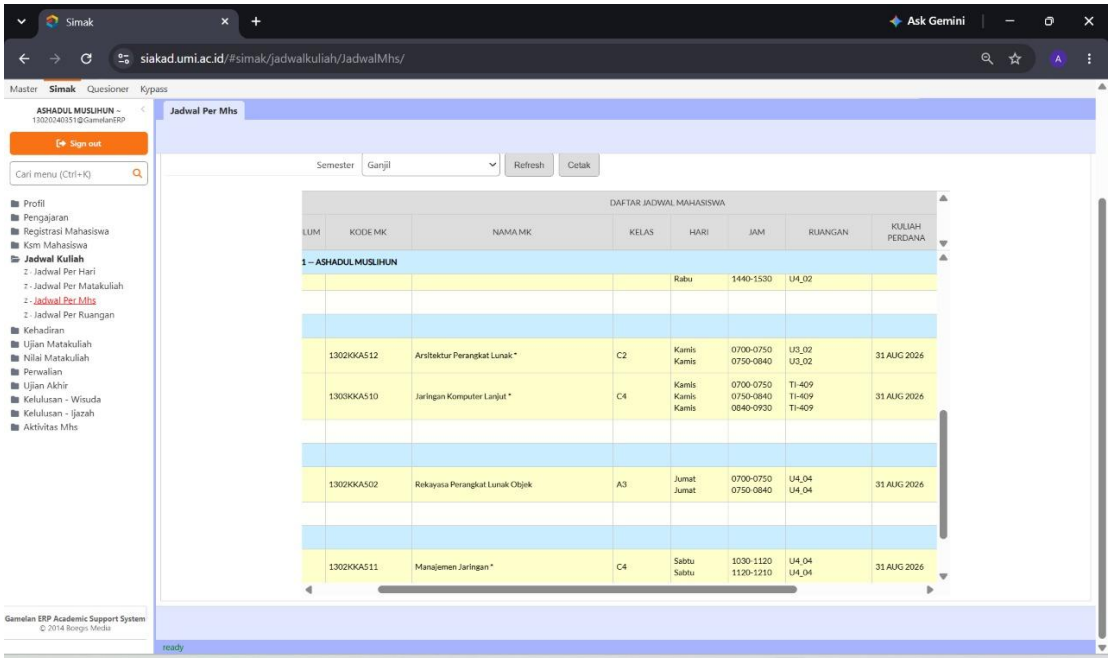
B. Bukti Empiris

1. Ringkasan Wawancara Responden 1 (Mahasiswa):

"Informasi perubahan jadwal atau perpindahan ruang biasanya saya terima terlambat, lewat grup chat atau dari teman, bukan dari SIAKAD. Bentrok jadwal juga baru ketahuan setelah jadwal dipublikasikan atau ketika perkuliahan hendak dimulai, karena sistem tidak memberikan notifikasi apa pun."

2. Ringkasan Wawancara Responden 2 (Pengelola Ruang / Fakultas): Wawancara ini memperkuat bukti empiris bahwa verifikasi ketersediaan ruang belum terintegrasi secara otomatis dan sinkron (sesuai ADR-001), dibuktikan dengan masih ditemukannya ketidaksesuaian data fisik di lapangan dengan sistem digital serta kendala jadwal bertabrakan.
3. Ringkasan Wawancara Responden 3 (Dosen): Wawancara dengan dosen memperkuat bukti empiris bahwa penyusunan jadwal kuliah dan alokasi ruangan di SIAKAD UMI belum terintegrasi secara otomatis dan sinkron (sesuai ADR-001). Hal ini dibuktikan dengan ditemukannya tiga masalah yang berulang: (1) jadwal mata kuliah yang bertabrakan, baik antar mata kuliah yang diampu dosen yang sama maupun dengan penggunaan ruangan oleh kelas lain; (2) ketidaksesuaian antara kapasitas ruangan dan jumlah mahasiswa, misalnya kelas berisi ±40 mahasiswa ditempatkan di ruangan berkapasitas ±30 orang, sehingga proses belajar mengajar tidak berjalan efektif; dan (3) perubahan ruangan atau jadwal yang baru diketahui dosen mendekati waktu perkuliahan, karena informasi di sistem tidak selalu sesuai dengan kondisi di lapangan. Akibatnya, dosen harus melakukan koordinasi manual (menghubungi staf, fakultas, atau mencari ruangan pengganti) yang memakan waktu dan mengganggu kegiatan mengajar.
4. Ringkasan Wawancara Responden 4 (Admin Akademik): Wawancara memperkuat bukti bahwa penyusunan jadwal perkuliahan masih rawan bentrok. Penyebab utamanya adalah mahasiswa yang mengambil mata kuliah lintas semester (ke atas atau ke bawah), keterbatasan kelas, dan jadwal lab yang tidak sinkron dengan jadwal teori, sehingga jadwal sering harus diubah berulang kali.

5. Bukti Penggunaan / Tangkapan Layar



1.3 Stakeholder Register dan Matriks Power/Interest

A. Stakeholder Register

1. Admin Akademik (Manajer Bisnis): Memiliki tanggung jawab mengatur jadwal seluruh prodi, memverifikasi permohonan perubahan, dan mengelola alokasi ruang. Menghadapi tantangan penyusunan jadwal akibat mahasiswa lintas semester, dan keterbatasan kelas.
2. Pengelola Ruang / Fakultas (Manajer Bisnis): Mengawasi ketersediaan dan status penggunaan ruang kelas fisik agar tidak tumpang tindih. Menghadapi ketidaksesuaian antara data fisik di lapangan dan data pada sistem digital.
3. Dosen (Pengguna Akhir): Membutuhkan kepastian jadwal mengajar yang akurat, tidak bentrok dengan agenda akademik lain, ruang dengan kapasitas yang sesuai jumlah mahasiswa, informasi perubahan jadwal/ruang yang tepat waktu, serta kemudahan dalam mengajukan *rescheduling*.
4. Mahasiswa (Pengguna Akhir): Membutuhkan tampilan jadwal pribadi yang konsisten, jelas, akurat, dan bebas bentrok di perangkat seluler maupun desktop, serta notifikasi langsung saat jam atau ruang berubah.

B. Kuadran Power / Interest

Power \ Interest	Interest Rendah / Sedang	Interest Tinggi
Power Tinggi	Pimpinan Akademik, Auditor Mutu *(Keep Satisfied)*	Admin Akademik, Pengelola Ruang / Fakultas *(Manage Closely)*
Power Rendah / Sedang	Tim Operasional / Helpdesk *(Monitor)*	Dosen, Mahasiswa (Keep Informed / Consult)

Keterlibatan stakeholder berdasarkan wawancara: wawancara dengan Pengelola Ruang/Fakultas dan Admin Akademik memperkuat posisi keduanya sebagai pihak yang harus dikelola secara intensif (*Manage Closely*), karena memegang kendali atas data dan keputusan alokasi jadwal serta ruang. Wawancara dengan Dosen memperkuat posisinya sebagai pengguna akhir dengan *Interest* tinggi dan *Power* rendah/sedang (*Keep Informed/Consult*): dosen tidak berwenang menetapkan jadwal dan ruang, tetapi terdampak langsung sehingga masukannya perlu dikonsultasikan, terutama soal kapasitas ruang, kepastian jadwal, dan kemudahan pengajuan *rescheduling*.

1.4 Architectural Significance Requirements (ASR)

- **ASR-01 (Akurasi Deteksi Bentrok - Functional Correctness):** Saat pengajuan perubahan atau penyusunan jadwal baru, sistem harus memvalidasi secara sinkron dan menolak 100potensi bentrok ruang, dosen, maupun jadwal mahasiswa (termasuk mahasiswa lintas semester dan jadwal laboratorium terhadap jadwal teori), serta memvalidasi kapasitas ruang terhadap jumlah mahasiswa terdaftar, sebelum data berhasil disimpan ke basis data.
- **ASR-02 (Ketersediaan Jam Sibuk - Performance & Availability):** Pada periode puncak (seperti minggu pertama perkuliahan dan pengisian KRS) dengan beban hingga 1.000pengguna bersamaan (*concurrent users*), sistem harus menjaga *availability* 99.9dengan waktu tanggap server (*response time*) $\leq$ 300 ms.
- **ASR-03 (Konsistensi Data Lintas Modul - Reliability & Data Integrity):** Setiap perubahan jadwal yang disetujui harus terintegrasi dan tercermin di seluruh tampilan dasbor (mahasiswa, dosen, serta informasi ruang) secara terpusat dalam waktu $<$ 5 detik, dengan data status ruang pada sistem yang selaras dengan kondisi fisik di lapangan.

1.5 Karakteristik Pendorong (Architectural Drivers)

Berdasarkan analisis kebutuhan dan dampak strukturalnya, ditetapkan 3 Karakteristik Pendorong Utama:

1. Konsistensi Jadwal dan Deteksi Bentrok (Utama):

- *Alasan:* Kesalahan jadwal berdampak langsung pada terhentinya operasional kegiatan belajar mengajar di kampus.
- *Fokus Arsitektur:* Penerapan mesin validasi (*validation engine*) sinkron dan transaksi atomik pada lapisan backend.

2. Keamanan Akses dan Auditabilitas (Utama):
   - *Alasan:* Jadwal merupakan data akademik penting; perubahan tanpa wewenang dapat merusak sistem perkuliahan. Seluruh riwayat perubahan harus dapat ditelusuri.
   - Fokus Arsitektur: Penerapan Role-Based Access Control (RBAC) pada setiap REST API endpoint serta asynchronous audit logging.
3. Kinerja Akses Jadwal (Utama):
   - *Alasan:* Terdapat lonjakan pengaksesan (*peak traffic*) secara masif pada awal semester.
   - *Fokus Arsitektur:* Penerapan strategi *read-caching* (Redis) dan indeks kueri basis data teroptimasi.

BAB II
SKENARIO

U2. Skenario Atribut Kualitas

2.1 Pendahuluan

Skenario atribut kualitas merupakan gambaran kebutuhan kualitas yang harus dipenuhi oleh Layanan *Academic Management* (khususnya sub-modul Jadwal Kuliah dan Ruang) pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI). Skenario ini disusun berdasarkan tiga *Architectural Significant Requirements* (ASR) pada U1, yaitu keakuratan deteksi bentrok (*Performance Efficiency*), kemudahan akses informasi jadwal (*Usability*), serta keandalan pencatatan dan konsistensi data (*Reliability*).

Tujuan penyusunan skenario ini adalah memastikan sistem mampu memberikan layanan yang cepat, mudah digunakan, dan dapat diandalkan dalam mengelola operasional penjadwalan perkuliahan.

2.2 Skenario Atribut Kualitas

1. ASR-1: Performance Efficiency (Performa & Deteksi Bentrok)

- **Sumber Stimulus:** Admin Akademik atau Dosen.
- **Stimulus:** Mengajukan penyusunan jadwal perkuliahan baru atau permohonan perubahan jadwal (*rescheduling*) secara dinamis.
- **Lingkungan:** Sistem berada dalam kondisi operasional normal maupun periode penyusunan jadwal di awal semester.
- **Artefak:** Modul Jadwal Kuliah dan Ruang (*Validation Engine* dan *Database Service*).
- **Respon:** Sistem melakukan verifikasi otomatis secara sinkron atas ketersediaan ruang, jam mengajar dosen, jadwal mahasiswa (termasuk lintas semester dan laboratorium terhadap teori), serta kapasitas fisik ruangan, lalu menolak transaksi jika ditemukan potensi bentrok sebelum data disimpan ke basis data.
- **Ukuran Respon:** Sistem menolak 100potensi bentrok (ruang, dosen, dan jadwal mahasiswa) serta kasus ketidaksesuaian kapasitas ruang, dengan durasi proses validasi transaksi backend ≤ 500 ms.

2. ASR-2: Usability (Kebergunaan)

- **Sumber Stimulus:** Mahasiswa dan Dosen (Pengguna Akhir).

- **Stimulus:** Mencari informasi jadwal pribadi, lokasi ruang kelas, perubahan jadwal (*rescheduling*), serta detail alokasi mata kuliah melalui perangkat seluler maupun komputer.
- **Lingkungan:** Sistem berada dalam kondisi operasional normal dan diakses secara harian oleh mahasiswa serta dosen.
- **Artefak:** Dasbor Jadwal Pribadi (*Personal Schedule Dashboard*), *API Gateway*, dan antarmuka pengguna (*Frontend Mobile/Web*).
- **Respon:** Sistem menampilkan tampilan jadwal secara visual yang jelas, terstruktur, konsisten, serta langsung mengirimkan notifikasi instan apabila terjadi perubahan jam atau ruang kepada mahasiswa dan dosen terkait.
- **Ukuran Respon:** Informasi jadwal ditampilkan dengan waktu tanggap kurang dari 1,5detik dan dapat ditemukan dalam maksimal 3kali klik.

3. ASR-3: Reliability (Keandalan)

- **Sumber Stimulus:** Admin Akademik / Sistem Backend yang memproses persetujuan perubahan jadwal.
- **Stimulus:** Menyetujui pengajuan perubahan jadwal atau perpindahan ruang yang berdampak pada pembaruan data secara masif di sistem.
- **Lingkungan:** Sistem sedang beroperasi aktif dan diakses oleh banyak pengguna secara bersamaan.
- **Artefak:** Layanan integrasi data terpusat, basis data relasional, dan modul *audit logging*.
- **Respon:** Setiap perubahan jadwal yang disetujui langsung disinkronkan, divalidasi, dan dicerminkan di seluruh tampilan dasbor secara terpusat, dengan data status ruang yang selaras dengan kondisi fisik di lapangan serta riwayat transaksi tercatat utuh.
- **Ukuran Respon:** Sistem mencapai keandalan pencatatan dan sinkronisasi sebesar 99,99dan memastikan data pembaruan jadwal yang berhasil diproses tidak hilang.

2.3 Kesimpulan

Sistem Informasi Akademik (SIAKAD) UMI pada layanan *Academic Management* harus mampu menangani validasi bentrok jadwal, menyediakan informasi jadwal dan perubahan yang mudah diakses, serta menjamin keandalan penyimpanan dan sinkronisasi data. Penerapan atribut *Performance Efficiency*, *Usability*, dan *Reliability* diharapkan dapat meningkatkan kualitas layanan akademik secara keseluruhan.

BAB III
TAKTIK DAN POLA

U3. TAKTIK ARSITEKTUR (Architectural Tactics)

Taktik arsitektur merupakan keputusan desain berskala kecil yang digunakan untuk mengontrol dan memenuhi atribut kualitas spesifik pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI), khususnya pada sub-modul Jadwal Kuliah dan Ruang. Berikut adalah penjelasannya berdasarkan tiga ASR utama.

3.1 Taktik Skenario 1: Performance Efficiency (Performa & Akurasi Deteksi Bentrok)

- **Nama Taktik:** Synchronous Validation & Atomic Transaction Control (Validate Input & Maintain Transaction Integrity)
- **Atribut Kualitas:** Performance Efficiency (Time Behaviour & Functional Correctness)
- **Cara Kerja:**
  - Saat Admin Akademik atau Dosen mengajukan penyusunan jadwal baru maupun perubahan jadwal (*rescheduling*), Web Backend tidak langsung menyimpan data ke tabel utama basis data.
  - Permintaan terlebih dahulu dilewatkan ke komponen *Validation Engine* secara sinkron. Mesin validasi ini mengeksekusi kueri pengecekan bentrok (*conflict detection query*) yang mencakup ketersediaan ruang, jam mengajar dosen, jadwal mahasiswa lintas semester, bentrok laboratorium vs. teori, serta batas kapasitas fisik ruangan.
  - Seluruh proses pengecekan dan eksekusi simpan dibungkus dalam satu mekanisme *Atomic Database Transaction* (ACID compliance). Jika ditemukan satu saja potensi bentrok atau *overcapacity*, transaksi dibatalkan (*rollback*) seketika dan pesan kesalahan spesifik dikembalikan ke pengguna.
- **Harga yang Dibayar (Trade-off / Cost):**
  - **Peningkatan Waktu Respon Penulisan (*Latency Overhead*):** Proses validasi sinkron yang ketat menambahkan *overhead* waktu komputasi backend sebelum data berhasil disimpan.
  - **Risiko *Database Lock Contraction*:** Kuncian transaksi (*row/table locking*) saat validasi masif dapat menyebabkan antrean singkat pada proses penulisan data jadwal lainnya jika dilakukan secara bersamaan.

3.2 Taktik Skenario 2: Usability (Kemudahan Akses Dasbor & Informasi Jadwal)

- **Nama Taktik:** Support User Initiative & Maintain Data Copies (Read-Model Caching & Instant Push Notification)
- **Atribut Kualitas:** Usability (Operability, Learnability, & User Responsiveness)
- **Cara Kerja:**
  - Untuk memastikan mahasiswa dan dosen dapat mengakses dasbor jadwal pribadi serta rincian alokasi ruang dalam waktu kurang dari 1,5 detik dan maksimal 3 kali klik, sistem menyalin data jadwal yang telah terbit dari basis data utama ke *Redis In-Memory Cache*.
  - Saat pengguna membuka dasbor via aplikasi seluler atau web frontend, sistem membaca langsung data siap pakai dari Redis tanpa perlu melakukan kueri *join* relasional yang berat.
  - Selain itu, saat terjadi pembaruan jadwal/ruang oleh admin, sistem memanfaatkan komponen *Notification Gateway* untuk mengirimkan notifikasi *push* instan secara otomatis ke perangkat mahasiswa dan dosen terdampak.
- **Harga yang Dibayar (Trade-off / Cost):**
  - **Overhead Invalidation Cache:** Memerlukan penerapan logika pembersihan cache (*cache invalidation logic*) yang presisi. Setiap kali jadwal diubah atau disetujui, sistem harus memvalidasi dan memperbarui cache Redis secara tepat waktu agar tidak menampilkan data usang (*stale data*).
  - **Biaya Infrastruktur Layanan Notifikasi:** Membutuhkan integrasi dan pemeliharaan *broker* notifikasi (seperti Firebase Cloud Messaging atau WebSocket server) untuk pengiriman notifikasi secara real-time.

3.3 Taktik Skenario 3: Reliability (Keandalan Pencatatan & Sinkronisasi Lintas Modul)

- **Nama Taktik:** State Persistence & Asynchronous Event Audit (Event-Driven Synchronization & Audit Logging)
- **Atribut Kualitas:** Reliability (Fault Tolerance, Data Integrity, & Auditability)
- **Cara Kerja:**
  - Saat pengajuan perubahan jadwal atau alokasi ruang disetujui oleh pihak berwenang, sistem menerbitkan status perubahan tersebut ke *Event Publisher/Message Broker*.
  - Komponen *Sync Worker Service* menerima *event* tersebut dan menyinkronkan status ruang serta dasbor perkuliahan di seluruh modul terkait secara terpusat dalam waktu < 5detik.
  - Pada saat yang sama, seluruh riwayat perubahan (siapa pengubah, waktu perubahan, ruang asal, dan ruang tujuan) dicatat secara *asynchronous* ke dalam

Audit Log Table tanpa mengganggu alur utama transaksi, guna menjamin keandalan pencatatan hingga 99,99% dan mencegah kehilangan jejak audit.

- **Harga yang Dibayar (Trade-off / Cost):**
  - **Kompleksitas Asinkronus (*Eventual Consistency*):** Terdapat jeda waktu sangat kecil (beberapa milidetik hingga < 5detik) antara persetujuan perubahan dengan konsistensi penuh tampilan di seluruh dasbor pengguna.
  - **Pertumbuhan Volume Data Log:** Pencatatan audit log secara menyeluruh memerlukan kapasitas penyimpanan basis data yang terus bertambah seiring tingginya frekuensi perubahan jadwal.

U4. POLA ARSITEKTUR (Architectural Patterns)

Pola arsitektur merupakan kerangka desain tingkat makro (*high-level architectural pattern*) yang mendefinisikan struktur organisasi sistem secara keseluruhan untuk mendukung taktik-taktik yang telah dipilih pada bagian U3.

3.1 Pola Utama yang Digunakan: *Layered Architecture* (N-Tier Architecture) dengan *Microservices-ready Modules*

- **Deskripsi Pola:** Sistem Informasi Akademik (SIAKAD) UMI pada layanan *Academic Management* memisahkan tanggung jawab sistem ke dalam beberapa lapisan logis yang jelas (*separation of concerns*):
  - **Presentation Layer:** Antarmuka web dan seluler tempat mahasiswa dan dosen mengakses *Personal Schedule Dashboard*.
  - **API Gateway & Business Logic Layer:** Lapisan penanganan routing, autentikasi berbasis RBAC, serta penempatan *Validation Engine* untuk memproses aturan bisnis penjadwalan secara ketat.
  - **Data & Caching Layer:** Lapisan penyimpanan relasional utama (MySQL dengan kendali ACID) yang dikombinasikan dengan *Redis In-Memory Cache* untuk mempercepat akses baca (*read operations*).

3.2 Kesesuaian Pola Arsitektur dengan Karakteristik Pendorong (ASR)

- **Mendukung ASR-01 (Functional Correctness & Validation):** Lapisan logika bisnis menggunakan pola terpusat pada *Validation Engine* untuk memastikan setiap transaksi perubahan jadwal mematuhi batasan aturan akademik secara atomik sebelum mencapai basis data.
- **Mendukung ASR-02 (Performance & Availability):** Integrasi *Redis Cache Layer* pada arsitektur data memungkinkan pembacaan dasbor jadwal secara masif tanpa membebani basis data relasional utama, menjaga waktu tanggap $\leq$ 300 msdi tengah lonjakan trafik pengguna.

- **Mendukung ASR-03 (Reliability & Data Integrity):** Penerapan basis data transaksional yang dipadukan dengan modul sinkronisasi asinkron memastikan integritas data status ruang dan catatan audit tetap terjaga sebesar 99,99.

BAB IV
DESAIN

U5. DESAIN ARSITEKTUR

4.1 Struktur Pandangan Arsitektur (*Architectural Views*)

Perancangan arsitektur sub-modul Jadwal Kuliah dan Ruang pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI) divisualisasikan melalui tiga pandangan utama:

1. Module View

Membagi elemen logis kode menjadi beberapa modul independen yang beroperasi secara terpusat:

- **Modul Penjadwalan Perkuliahan**: Mengelola draf jadwal, slot waktu, dan alokasi mata kuliah.
- **Modul Pengelolaan Ruang**: Mengelola master data ruang kelas, fasilitas fisik, serta status penggunaan ruang.
- **Modul Validasi Bentrok (*Validation Engine*)**: Mengisolasi seluruh aturan bisnis terkait pengecekan bentrok dosen, ruang, jam, mahasiswa lintas semester, serta kapasitas kelas.
- **Modul Notifikasi (*Notification Service*)**: Mengirimkan peringatan instan (*push notification*) kepada dosen dan mahasiswa apabila terdapat jadwal atau ruang yang berubah.
- **Modul Audit Logging**: Mencatat seluruh transaksi perubahan jadwal secara terpisah untuk kebutuhan penelusuran (*audit trail*).

2. Component & Connector (C&C) View

Menunjukkan interaksi antar-komponen sistem secara *runtime*:

- **API Gateway**: Bertindak sebagai *single entry point* yang menangani autentikasi JWT dan pembatasan hak akses (*Role-Based Access Control* / RBAC).
- **Validation Engine Component**: Berinteraksi secara sinkron dengan *backend* aplikasi saat pengajuan perubahan jadwal dilakukan.
- **Redis Cache**: Melayani kueri pembacaan dasbor perkuliahan secara langsung tanpa membebani basis data utama.

- **Message Broker (RabbitMQ)**: Mengelola antrean *event* secara asinkron untuk sinkronisasi data lintas modul dan pengiriman notifikasi otomatis.

3. Allocation View

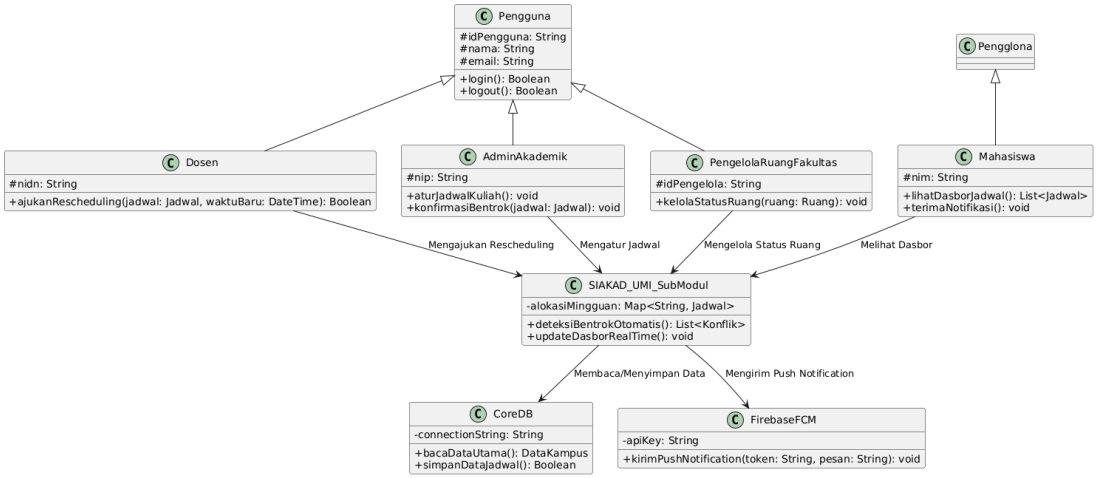
Memetakan komponen perangkat lunak ke dalam infrastruktur fisik dan virtual Kampus Universitas Muslim Indonesia (UMI):

- **Web/Application Server (Node.js/Go)**: Ditempatkan pada *Virtual Private Server* (VPS) kampus yang terhubung dengan *Load Balancer*.
- **In-Memory Cache Server (Redis Cluster)**: Dialokasikan khusus pada server memori tinggi (*high memory*) untuk melayani *caching* data jadwal.
- **Database Server (MySQL Cluster)**: Ditempatkan pada server terpisah dengan konfigurasi *Master-Slave Replication* dan penyimpanan SSD berkecepatan tinggi di Data Center UMI.

4.2 Diagram Konteks (C4 System Context - Level 1)

Diagram Konteks Sistem menggambarkan batas operasional sub-modul Jadwal Kuliah dan Ruang SIAKAD UMI, aktor yang berinteraksi dengannya, serta keterhubungan dengan sistem eksternal pendukung.

Gambar :



BAB V
EVALUASI DAN ADR

U6. EVALUASI ARSITEKTUR

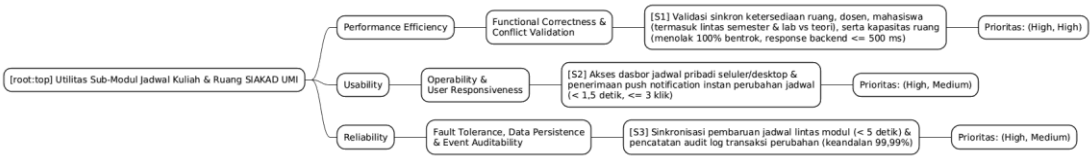
Dokumen ini memuat evaluasi menyeluruh terhadap rancangan arsitektur sub-modul Jadwal Kuliah dan Ruang pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI). Evaluasi dilakukan melalui tiga metode utama: Pohon Utilitas Mini, Matriks Evaluasi Trade-off Berbobot, dan Analisis ATAM (*Architecture Trade-off Analysis Method*), serta diakhiri dengan penjelasan Keterbatasan Masalah Utama.

6.1 Pohon Utilitas Mini (*Mini Utility Tree*)

Pohon Utilitas Mini digunakan untuk memetakan dan memprioritaskan kebutuhan kualitas arsitektur sistem berdasarkan tiga *Architectural Significant Requirements* (ASR) pada U1 dan U2 (yaitu **Performance Efficiency / Functional Correctness**, **Usability**, dan **Reliability**).

Setiap skenario dinilai berdasarkan dua dimensi prioritas:

- **Importance to Business (H/M/L)**: Tingkat kepentingan skenario bagi operasional akademik SIAKAD UMI (*High, Medium, Low*).
- **Difficulty to Implement (H/M/L)**: Tingkat kesulitan teknis dalam mengimplementasikan solusi arsitektur (*High, Medium, Low*).



6.2 Matriks Evaluasi Trade-off Berbobot (*Weighted Trade-off Evaluation Matrix*)

Matriks ini mengukur sejauh mana keputusan arsitektur (taktik dan pola yang dipilih pada U3 & U4) mendukung pencapaian ASR serta mengidentifikasi dampak *trade-off* terhadap atribut kualitas lainnya.

Arsitektur / Taktik Solusi	**Terkait ASR**	**Manfaat Utama (*Benefit*)**	**Dampak *Trade-off* (*Cost / Risk*)**	**Bobot Prioritas**	
Synchronous Validation & Atomic	**ASR-01**	Menjamin 100% validasi bentrok ruang,	Menimbulkan *overhead* waktu	**45%**	**4.8**

Transaction Control		dosen, mahasiswa lintas semester, dan kapasitas secara presisi sebelum *commit*.	komputasi (*latency overhead*) dan potensi *database locking* saat transaksi masif bersamaan.		
Read-Model Caching (Redis) & Instant Push Notification	**ASR-02**	Menyajikan dasbor jadwal pribadi dalam $<$ 1,5 detiktanpa membebani basis data utama dan memberikan notifikasi perubahan *real-time*.	Memerlukan logika pembersihan memori (*cache invalidation*) yang rumit agar tidak ada *stale data*, serta biaya integrasi push notification.	**30%**	**4.6**
Event-Driven Synchronization & Asynchronous Audit Logging	**ASR-03**	Memastikan integritas data status ruang sinkron dalam < 5 detikdan riwayat transaksi tercatat utuh tanpa mengganggu alur utama.	Adanya jeda waktu sangat kecil (*eventual consistency*) dan pertumbuhan volume basis data *audit log* seiring waktu	**25%**	**4.5**

Total Weighted Score: $(0.45 \times 4.8) + (0.30 \times 4.6) + (0.25 \times 4.5) = \mathbf{4.665} / \mathbf{5.00}$

6.3 Analisis ATAM (Architecture Trade-off Analysis Method)

Analisis ATAM mengevaluasi keputusan arsitektur berdasarkan empat elemen kunci: Trade-off, Sensitivity Points, Risks, dan Non-Risks.

1. Scenario Validation: SC-01 (Akurasi Deteksi Bentrok Sinkron)

- **Sensitivity Point**: Kecepatan kueri validasi bentrok sangat sensitif terhadap kompleksitas indeks basis data dan struktur relasi antara jadwal teori, laboratorium, dosen, dan mahasiswa lintas semester.

- **Trade-off**: Mengorbankan *write latency* (kecepatan simpan) demi menjamin *data accuracy* dan konsistensi 100% bebas bentrok.
- **Risk**: Jika terjadi pengajuan perubahan jadwal secara masif oleh ratusan admin prodi pada saat yang bersamaan, *row-level locking* dapat menyebabkan antrean transaksi (*bottleneck* penulisan).
- **Non-Risk**: Penggunaan *atomic transaction rollback* menjamin tidak akan ada data setengah tersimpan yang menyebabkan jadwal bentrok lolos ke sistem.

2. Scenario Validation: SC-02 (Performa Dasbor & Notifikasi)

- **Sensitivity Point**: Kecepatan *response time* dasbor sangat bergantung pada efisiensi *caching* di Redis In-Memory Store.
- **Trade-off**: Mengorbankan sedikit penggunaan *RAM Server* dan penambahan infrastruktur *cache* demi mencapai *response time* dasbor $< 1{,}5$ detik.
- **Risk**: Kegagalan *invalidation cache* saat admin mengubah jadwal dapat menyebabkan mahasiswa melihat jadwal lama (*stale data*) di aplikasi.
- **Non-Risk**: Penerapan API Gateway menjamin autentikasi JWT dan RBAC tetap berjalan aman tanpa membebani Redis.

3. Scenario Validation: SC-03 (Konsistensi Data Lintas Modul & Audit Log)

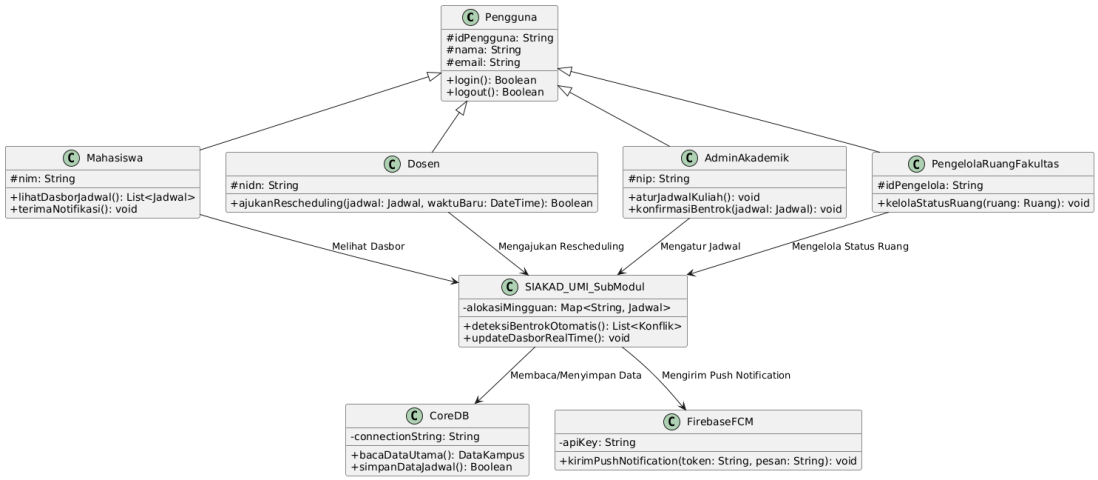
- **Sensitivity Point**: Konsistensi data status ruang di seluruh modul terpengaruh oleh *throughput* Message Broker (RabbitMQ).
- **Trade-off**: Mengorbankan konsistensi seketika (*strict immediate consistency*) menjadi *eventual consistency* (< 5 detik) demi menjaga *throughput* sistem backend tetap tinggi.
- **Risk**: Apabila Message Broker mengalami pemadaman (*down*), pengiriman notifikasi instan dan pencatatan *audit log* dapat tertunda hingga service pulih.
- **Non-Risk**: Penyimpanan data utama di MySQL Cluster Master-Slave menjamin data jadwal yang telah *commit* tidak akan hilang (*durability*).

6.4 Keterbatasan Masalah Utama

Meskipun rancangan arsitektur sub-modul Jadwal Kuliah dan Ruang SIAKAD UMI ini telah dirancang secara optimal untuk mengatasi isu bentrok dan kinerja, terdapat beberapa keterbatasan operasional dan teknis yang perlu diperhatikan:

1. **Dependensi Kualitas Input Data Master Ruang Fisik**: Meski sistem memiliki *Validation Engine* yang presisi, keakuratan validasi kapasitas dan status ruang sangat bergantung pada kedisiplinan Pengelola Ruang/Fakultas dalam memperbarui master data ruang fisik (misalnya perubahan tata letak kursi atau perbaikan fasilitas fisik di lapangan).
2. **Tantangan Eventual Consistency pada Notifikasi Real-Time**: Penerapan arsitektur asinkron berbasis *Message Broker* menyisakan jeda waktu kecil (< 5 detik) dalam sinkronisasi dasbor dan pengiriman push notification. Dalam kondisi koneksi jaringan pengguna akhir yang tidak stabil, mahasiswa mungkin mengalami keterlambatan penerimaan notifikasi.
3. **Pertumbuhan Ukuran Basis Data Audit Log**: Pencatatan *audit trail* secara mendalam untuk setiap transaksi perubahan jadwal akan meningkatkan beban kapasitas media penyimpanan (*disk storage*) pada Database Server MySQL Cluster secara signifikan seiring berjalannya semester.

6.5 Diagram Konteks System Context (PlantUML)



U7. ARCHITECTURAL DECISION RECORDS (ADR)

Dokumen *Architectural Decision Record* (ADR) ini mencatat keputusan-keputusan arsitektur penting yang diambil dalam perancangan Sub-Modul Jadwal Kuliah dan Ruang pada Sistem Informasi Akademik (SIAKAD) Universitas Muslim Indonesia (UMI). Setiap ADR memuat konteks permasalahan, keputusan yang ditetapkan, serta konsekuensi teknis yang ditimbulkan.

ADR-001: Implementasi Validation Engine Sinkron & Message Queueing (RabbitMQ) untuk Deteksi Bentrok & Sinkronisasi Notifikasi

- **Title**: ADR-001: Penerapan Validation Engine Sinkron dengan Atomic Transaction Control serta Message Queueing berbasis RabbitMQ untuk Sinkronisasi Jadwal dan Notifikasi Asinkron
- **Status**: Accepted
- **Context**: Proses penyusunan dan pengajuan perubahan jadwal perkuliahan (*rescheduling*) pada SIAKAD UMI sebelumnya rawan memicu bentrok jadwal mata kuliah, ruang kelas, jadwal laboratorium vs teori, waktu mengajar dosen, serta ketidaksesuaian kapasitas ruang terhadap jumlah mahasiswa terdaftar (ASR-01). Selain itu, perubahan jadwal yang disetujui berpotensi memicu lonjakan beban jaringan (*traffic spike*) saat ribuan mahasiswa dan dosen mengakses dasbor secara serentak pada periode puncak/awal semester (ASR-02). Jika pemrosesan perubahan dan pengiriman notifikasi dilakukan secara langsung (*synchronous write & push*), sistem berisiko mengalami *HTTP 504 Gateway Timeout* dan ketidaksinkronan data status ruang.
- **Decision**:
  i. Mengimplementasikan **Validation Engine secara sinkron** di lapisan backend yang dibungkus *Atomic Database Transaction (ACID)* untuk memvalidasi 100% ketersediaan ruang, jam mengajar dosen, jadwal mahasiswa (termasuk lintas semester dan lab vs teori), serta kapasitas fisik ruangan sebelum transaksi disimpan (*commit*) ke basis data utama.
  ii. Mengimplementasikan **RabbitMQ sebagai Message Broker** dengan pola *Event-Driven Architecture (EDA)*. Setiap transaksi perubahan jadwal yang berhasil disetujui akan menerbitkan *event* ke antrean asinkron (*message queue*) untuk diproses oleh *background worker* guna menyinkronkan data lintas modul dan mengirimkan *push notification* via Firebase Cloud Messaging (FCM).
- **Consequences**:
  - **Positif**: Terhindar 100% dari potensi jadwal bentrok dan *overcapacity* ruang kelas sebelum data tersimpan, sistem terbebas dari risiko *crash* saat lonjakan akses jadwal di awal semester, waktu tanggap *frontend* dasbor tetap konsisten ≤ 300 ms, dan mampu menampung 1.000+ pengguna serentak dengan *downtime* 0% (*availability* 99,9%).

- **Negatif**: Proses validasi sinkron yang ketat menambah sedikit *overhead* waktu komputasi penulisan (*latency overhead* backend ≤ 500 ms), serta terdapat jeda waktu sangat kecil (*eventual consistency delay* < 5 detik) sebelum seluruh pengguna menerima notifikasi konfirmasi perubahan jadwal secara resmi.

- ADR-002: Separasi Read-Model (Redis Caching) dan Relational Database Transaction (ACID) untuk Layanan Jadwal SIAKAD UMI
- Title: ADR-002: Penggunaan Redis In-Memory Caching untuk Pembacaan Dasbor Jadwal Pribadi & Transaksi RDBMS (ACID) untuk Manajemen Ruang dan Audit Logging
- Status: Accepted
- Context: Sistem membutuhkan performa pencarian dan penayangan informasi dasbor jadwal pribadi (*Personal Schedule Dashboard*) bagi mahasiswa dan dosen yang cepat ($< 1{,}5$ detik dan ≤ 3 kali klik) pada perangkat seluler maupun desktop (ASR-02). Di sisi lain, sistem harus menjamin keandalan pencatatan transaksi perubahan jadwal, konsistensi status penggunaan ruang kelas fisik, serta *audit trail* agar tidak ada riwayat perubahan yang hilang (*reliability* 99,99%) (ASR-03).
- Decision:
  i. Menggunakan Redis In-Memory Caching untuk melayani kueri pembacaan dasbor jadwal perkuliahan mingguan dan ketersediaan ruang secara terpisah (*read-model*), sehingga *frontend* membaca data langsung dari *cache* tanpa melakukan *query join* relasional yang berat.
  ii. Menggunakan Relational Database Management System (RDBMS - MySQL Cluster) dengan dukungan transaksi ACID untuk modul validasi jadwal, alokasi master ruang, dan pencatatan *audit log* transaksi perubahan.
- Consequences:
  - Positif: Dasbor jadwal pribadi dan rincian alokasi ruang dapat diakses secara instan dengan waktu tanggap $< 1{,}5$ detiktanpa membebani basis data relasional utama. Data transaksi perubahan jadwal dan *audit log* tersimpan secara permanen, aman, dan terstruktur tanpa risiko kehilangan data (*persistence & reliability* 99,99%).
  - Negatif: Memerlukan penerapan logika pembersihan memori (*cache invalidation logic*) yang presisi pada Redis setiap kali terjadi persetujuan perubahan jadwal atau penyesuaian status ruang fisik oleh admin/pengelola ruang, guna mencegah tampilan data lama (*stale data*).

BAB VI
PENUTUP

A. Daftar Pustaka

[1] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston: Addison-Wesley Professional, 2021.

[2] N. Richards and M. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA: O'Reilly Media, 2020 (relevan dan banyak dijadikan rujukan standar industri hingga tahun 2025/2026).

[3] A. Fernandes, R. Santos, and M. Silva, "Performance evaluation of event-driven architecture versus API-driven architecture in microservices-based systems," IEEE Access, vol. 11, pp. 45210–45222, 2023.

[4] K. Sharma and R. Kumar, "Optimizing data access and fault tolerance in distributed cloud-native microservice architectures using in-memory caching," IEEE Transactions on Services Computing, vol. 17, no. 4, pp. 1820–1835, Jul./Aug. 2024.

[5] D. M. Patel and S. V. Gohil, "Systematic literature review on scalability, reliability, and security challenges in cloud-native software ecosystems," Journal of Systems and Software, vol. 212, p. 112015, Jun. 2025.